FOUNDER-ONLY BRIEFING · OCTOBER 2026

Meeting Neysa's CTO

Your product under the hood, every recommendation it makes, and how it could fit with Neysa Velocis. Skim page 1; the rest is reference.

Read this first

- **The pitch:** “Command Center shows what's happening. We tell each team what to change, what it saves in ₹, and what it costs them in latency.” You're a decision layer on their data layer, not a second monitoring tool.
- **What it is:** a read-only Helm chart, inside the customer's Kubernetes cluster. It reads NVIDIA DCGM metrics, links each GPU to its pod, classifies waste with fixed rules and prices it at the customer's own ₹ rates. It recommends; it never acts.
- **Evidence:** live on GKE (L4) and a k3s RTX 3090 cluster (full pilot rehearsal, all 6 checks passed); Microsoft Philly trace (15.6% of GPU-hours with a headline recommendation); Azure LLM trace (19,366/19,366 requests served); right-sizing tested on real GPUs.
- **The ask:** a free pilot on ONE cluster: one of Neysa's own managed-service clusters, or one willing VKE customer. Pilot agreement signed first.

Watch out

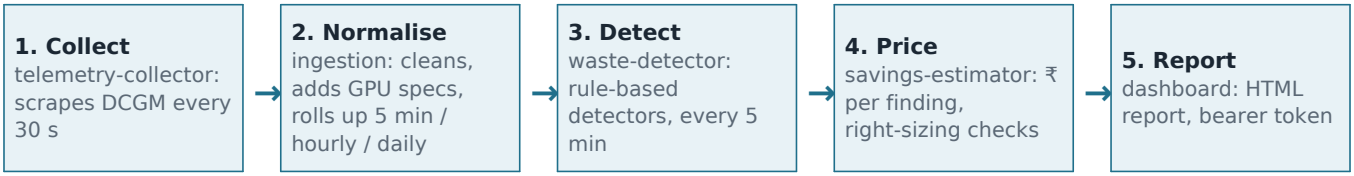
- **H200, B200 and GH200 are not recognised yet.** Findings on them show no ₹ figure. H100, L40S and L4 work. Don't demo on H200. About a 30-minute fix plus tests, before any pilot.
- **Don't share internals before an NDA.** They collect the same DCGM data and could build this. Talk outcomes and evidence; hold back thresholds and sizing formulas.
- **Velocis VMs, bare metal and Slurm aren't supported** (Kubernetes only). Don't promise them.
- **Expect the conflict question:** “you tell our customers to use fewer GPUs.” Answers on page 6.

Ask them

- Does Command Center make recommendations today, or only show data? Is it on the roadmap?
- How do customers split between VKE, VMs, bare metal and Slurm?
- What's utilisation on your own managed inference fleet? Would you pilot it there?
- Are you capacity-constrained? Would freed GPUs be resold?

1. Under the hood

Five services in **one pod** sharing one 5 GiB volume. Each has its own SQLite database, reads the previous stage's database read-only and writes only its own. Python, shipped as compiled native code (no source in the images).



DCGM exporter (one per GPU node) and kube-state-metrics feed stage 1.

What it measures

- **DCGM fields:** GPU utilisation, framebuffer memory used and free, power draw, SM clock.
- **“Utilisation” is busy time:** DCGM GPU_UTIL is the share of time any kernel runs, not compute used. An LLM server reads 70–100% whenever one request is in flight. That's why the average-load detector exists.
- **GPU-to-pod link:** dcgm-exporter reads the kubelet pod-resources socket (namespace, pod, container). The **workload** (Deployment/StatefulSet/Job) is derived from the pod-name pattern, so history survives rescheduling. Honest detail: owner references aren't read yet; kube-state-metrics is scraped but not used yet.
- **Per-node scraping** through a headless Service, so every node's exporter is read.

Processing

- Bad values (NaN, out of range) rejected. Each GPU model matched to FP16 TFLOPS, memory bandwidth and memory size. MIG instances tracked separately.
- Rolled up into 5-minute, hourly and daily summaries per GPU per workload.
- **Coverage:** every hour records the share of expected scrapes that arrived. Detectors ignore hours under 80% coverage: missing data is “unknown”, never “idle”.
- Retention: raw 14 days, cleaned samples 3 days, summaries and findings 90 days.

Running it, and security

- **Install:** helm install into one namespace, ~15 min. ~0.2 CPU and ~0.6 GiB requested for the main pod.
- **Permissions:** get/list/watch on pods, nodes, namespaces only. The collector has no Kubernetes API token.
- **Hardening:** non-root, read-only file systems. Report via kubectl port-forward with a rate-limited bearer token.
- **No privileged pod** if the cluster already runs dcgm-exporter (GPU Operator): it reuses it.
- **No data leaves the cluster**, except an optional alert webhook the customer configures.
- Liveness checks on every service; webhook alert when a service keeps failing.

Evidence you can quote

Test	Result
Live clusters	GKE (L4); k3s RTX 3090 pilot install rehearsal: all 6 checks passed, survived a node reboot, upgraded in place with data intact, found and fixed a real bug
Microsoft Philly training trace	15.6% of GPU-hours carry a headline recommendation
Azure LLM inference trace	19,366/19,366 requests served; over-provisioning finding fired
Right-sizing on real GPUs	Compute-only sizing failed (RTX 3090 → T4 was 2.3x slower vs 1.1x predicted). Fixed with the bandwidth check; the fix held on an RTX 4060 Ti (predicted up to 3.25x, measured 1.72x)
Automated tests	351 tests, run in CI against the compiled code

2. Findings and recommendations

Every threshold below is a Helm setting the customer can change. An hour counts only with ≥ 80% coverage; a finding is priced only with average coverage ≥ 0.7.

How findings are grouped in the report

Group	Findings	In the headline?
Already wasted (observed)	Idle	Yes: money already spent
Monthly, if usage continues (projected)	Over-provisioned; Oversized by average load; Recurring quiet hours	Yes
Review candidates (upper bound)	Fragmentation; Multi-GPU job could use fewer GPUs	No, shown separately
Detected, no recommendation yet	Under-use; GPU time with no pod	No, shown last

Headline counting: one action per GPU — alternatives on the same GPU aren't added; the GPU's largest estimate counts. Recurring quiet hours add up with each other, then compete as one alternative. Idle amounts are summed (separate windows of money already spent).
No ₹ figure (“none -- not priceable”) when: the GPU model isn't in the spec table (today H200, B200, GH200); there's no rate; the recommended GPU has no rate; or it's right-sizing on a MIG slice.

Every finding at a glance

Finding	Fires when	Recommendation (exact text)	₹ figure
1. Idle GPU	Average **and** peak ≤ 2% in every hour, for ≥ 4 consecutive hours. Any burst breaks it.	“terminate or reclaim this idle GPU allocation”	Idle hours × ₹/hr. Already spent, not projected.
2. Over-provisioned	Peak ≤ 30% over ≥ 24 observed hours.	“right-size to <GPU> (<cautions>)”	(current ₹/hr − target ₹/hr) × 730 h/month
3. Oversized by average load (LLM servers)	Peak > 30% but busiest hour's average ≤ 30%, over ≥ 24 h.	Same “right-size to …”, sized against a stricter 50% target	Same formula as 2
4. Recurring quiet hours	Same hour of day (UTC) ≤ 5% on ≥ 80% of ≥ 7 observed days.	“scale down or scale to zero during this recurring low-usage window”	Share of quiet days × 30 days × ₹/hr, per hour slot; slots add up
5. Fragmentation (review)	≥ 2 GPUs on one node, from ≥ 2 workloads, each ≤ 50% at once, for ≥ 4 h.	“review whether this workload can share a GPU … (needs GPU sharing via time-slicing, MPS, or MIG; memory fit not checked)”	Rate × (1 − avg util) × 50% capture × 730. Upper bound.
6. Multi-GPU job could use fewer GPUs (review)	Job's combined busy time fits on fewer GPUs at 70% each, for ≥ 4 h.	“review whether this multi-GPU job can run on fewer GPUs (check memory fit, batch size and run time; …)”	Rate × fraction releasable × 50% × 730, per GPU. Upper bound.
7. Under-used, no recommendation	Attributed GPU-hour below the 70% target, not covered by any other finding.	“none yet -- detected waste without a reliable recommendation; review manually”	Rate × shortfall below 70% × hours. Already spent.
8. GPU time with no pod	Same as 7, for GPUs not linked to any pod.	Same as 7; says “unassigned GPU on a running node or work running outside Kubernetes”	Same as 7

3. The right-sizing engine (findings 2 and 3)

- **Need:** current capacity × observed load ÷ target (70%; 50% for the average-load finding), computed for both **FP16 compute** and **memory bandwidth**.
- **Memory:** if peak memory use is known, the target's usable memory (93% of spec) must hold the peak + 10%.
- **Pick:** the smallest datacenter GPU that passes every check. Consumer GPUs are never suggested as targets.
- **Slowdown caution:** the larger of the compute and bandwidth drops: “each request may take up to ~Nx longer -- fine for batch/offline work, check latency targets before moving an interactive service”.
- **“memory fit not checked”** when the workload holds ≥ 85% of GPU memory (vLLM/TensorFlow pre-allocate) or there's no memory data.
- **When nothing fits:** “none -- no smaller known SKU meets the SLA ceiling” at ₹0.

Real outputs from the engine (A100 80GB at ₹300/hr)

Situation	Target	Cautions	Monthly saving
Peak 20%, no memory data	A10	~3.4x slower; memory fit not checked (24 GB vs 80 GB)	(₹300 − ₹105) × 730 = ₹1,42,350
Peak 20%, peak memory 18 GiB	A10	~3.4x slower; memory checked (needs 19.8 GiB, fits)	₹1,42,350
Peak 20%, holds 92% of memory (vLLM)	A10	~3.4x slower; memory fit not checked (pre-allocation)	₹1,42,350
Busiest-hour average 25% (finding 3)	A100 40GB	~1.3x slower; memory fit not checked (40 GB vs 80 GB)	(₹300 − ₹220) × 730 = ₹58,400

Example idle: 6 h on an A100 80GB at ₹300 = ₹1,800 already wasted. Example quiet hours: 90% of days at 03:00 on an L4 at ₹70 = 27 h × ₹70 = ₹1,890/month.

Limits a CTO might probe

- **“Smallest” is by specs, not price.** With some rate cards the target costs the same or more and the saving shows ₹0 (e.g. RTX 3090 → T4).
- **The slowdown is an upper bound from specs**, not measured. In our real test: predicted up to 3.25x, measured 1.72x.
- **No right-sizing on MIG slices:** DCGM doesn't report slice size, so it says “not priceable” rather than guess.
- **Recurring patterns use UTC hours** (03:00 UTC = 08:30 IST).
- **Known cosmetic issue:** a 0%-peak idle GPU also appears under Over-provisioned at ₹0 with “none”. Logged, to fix.
- **H100 NVL** is matched to H100 SXM specs (understates memory: 94 GB, not 80 GB).

4. How it fits with Neysa Velocis

What Velocis already has (public sources)

- **Command Center:** real-time GPU, disk and NVMe utilisation; GPU health via DCGM; custom metrics; role-based access; per-team cost tracking. An add-on with Velocis instances.
- **Overwatch** is network observability; **Aegis** is model security. Neither overlaps with you.
- **Where workloads run:** GPU VMs, containers, VKE (customer-owned Kubernetes), bare metal (Elite BMaaS), Slurm. GPUs include H100, H200, L40S, L4.
- From search summaries of Neysa's pages (couldn't open them directly). Confirm on the call.

Overlap and gap

Capability	Velocis Command Center	Your product
GPU utilisation and health (DCGM)	Yes	Yes (same data source)
Spend per team or project	Yes	Partly (₹ per finding and per workload)
Classifies why a GPU is wasting money	Not mentioned	Yes
Names a smaller GPU, checked for compute, bandwidth, memory, with stated slowdown	Not mentioned	Yes, tested on real GPUs
Catches oversized LLM servers that look 100% busy	Not mentioned	Yes
Prices each recommendation in ₹	Not mentioned	Yes

Four ways it could fit, easiest first

Option	What happens	Gaps / work	Commercially
A. Add-on for VKE customers (works today)	Customers install the Helm chart in their VKE cluster; it can reuse their DCGM exporter. Neysa's price list as default rates.	H200/B200/GH200 support (~30 min).	Marketplace listing, revenue share
B. Neysa's own fleet	Run on clusters behind their managed Inference-as-a-Service and AI PaaS: idle GPUs there cost Neysa, so the case is simplest.	H200 support; maybe scale (see C).	Internal licence
C. Built into Command Center	Engine reads metrics Neysa already collects; findings appear in their dashboard.	Prometheus/Thanos input adapter; JSON/REST output (today HTML only); per-tenant isolation; fleet-scale database (today SQLite in one pod); SSO/RBAC. ~2–4 months solo [estimate].	Annual OEM / white-label licence
D. VMs, bare metal, Slurm	Not supported today (roadmap #2): metrics visible but no link to the job using the GPU.	Don't promise it.	—

The conflict question, and your answers

“A GPU cloud earns more when customers use more GPUs — you tell them to use fewer.”

- **Neysa sells on cost** (40–60% better unit economics than hyperscalers). Right-sizing H100 → L40S/L4 within Neysa keeps the customer; one who feels overcharged leaves.
- **GPU capacity is scarce in India.** Freed GPUs resell to IndiaAI-driven demand: more revenue per GPU.
- **On their managed services (option B), waste is Neysa's own cost.**

Protect your IP: share vs. hold back

Fine to share now	Hold back until an NDA / pilot agreement
What it finds; sample findings and recommendations; validation results (3090 → T4 test, rehearsal, Philly, Azure); install, read-only, data stays in cluster	Detector thresholds; right-sizing formulas; the 85% pre-allocation rule; the 93% usable-memory figure; how the headline is counted — the IP the compiled images protect (layer 2 in docs/IP_PROTECTION.md)

Questions to ask

- Does Command Center make recommendations today, or only show data? On the roadmap?
- How do customers split between VKE, VMs, bare metal and Slurm? (Decides whether option A is big enough.)
- Do customers' VKE clusters run a DCGM exporter by default?
- What's utilisation on your own managed inference fleet? Would you pilot option B?
- Are you capacity-constrained? (Decides whether freed GPUs are good news.)
- Marketplace add-on, or embedded in Command Center?

What to propose

A free pilot on **one** cluster — one of Neysa's internal managed-service clusters (B) or one willing VKE customer (A). Pilot agreement signed first; H200 support added before it starts.

Sources: neysa.ai/neysa-velocis ; neysa.ai/pricing (Command Center add-on) ; neysa.ai/blog/ai-acceleration-cloud-system ; getinfra.cloud/providers/neysa ; enterpriseitworld.com (Neysa platforms) ; Neysa careers page (Overwatch) ; analyticsindiamag.com (Neysa funding). Product facts checked against the code on 2026-10-05.